



Docker 컨테이너 기초

컨테이너 기술의 이해부터 Docker Compose까지

 5시간 과정

 입문자 대상

 개념·이론 중심

컨테이너 · 이미지 · 라이프사이클 · Dockerfile · 볼륨 · 네트워크 · Compose

COURSE OVERVIEW

강의 구성 — 5시간 로드맵

각 파트는 약 60분 구성이며, 앞 파트의 개념 위에 다음 파트가 쌓이는 구조입니다

1	컨테이너와 가상화 왜 컨테이너인가 · VM vs 컨테이너 · 리눅스 커널 기술		60분
2	Docker 아키텍처와 이미지 Docker 구조 · 이미지 레이어 · 이미지와 컨테이너의 관계		60분
3	컨테이너 라이프사이클 상태 전이 · run의 실체 · 주요 관리 명령어		60분
4	Dockerfile과 이미지 빌드 Dockerfile 문법 · 빌드 캐시 · 멀티스테이지 · 경량화		60분
5	데이터 · 네트워크 · Compose 볼륨과 마운트 · 도커 네트워크 · Docker Compose		60분

PART 1

컨테이너와 가상화

컨테이너는 왜 등장했고, VM과 무엇이 다른가

- "내 컴퓨터에서는 되는데요" — 환경 문제
- 가상화 기술의 발전 과정
- VM과 컨테이너의 구조 비교
- 컨테이너를 만드는 리눅스 커널 기술

01



 60분

"제 컴퓨터에서는 잘 되는데요?"

소프트웨어 배포의 오래된 문제 — 실행 환경의 불일치

개발과 운영 사이의 간극

환경 불일치

개발 PC, 테스트 서버, 운영 서버의 OS·라이브러리 버전이 제각각

의존성 충돌

한 서버에 여러 앱을 올리면 서로 다른 버전의 런타임이 충돌

배포의 복잡성

설치 문서 따라 수작업 구성 — 느리고, 사람마다 결과가 다름

컨테이너의 해법 — 실행 환경까지 통째로 포장

해운 컨테이너에서 온 아이디어

- 내용물이 무엇이든 규격이 같으면 어떤 배·트럭·크레인으로도 운송 가능
- 소프트웨어도 앱 + 라이브러리 + 설정을 하나의 규격 상자에 포장
- 어디서 실행해도 동일하게 동작 — "Build once, Run anywhere"

컨테이너(Container)란?

애플리케이션과 그 실행에 필요한 모든 것(코드, 런타임, 라이브러리, 설정)을 하나로 묶어, 격리된 공간에서 실행하는 기술

가상화 기술의 발전

물리 서버에서 컨테이너까지 — 더 가볍고, 더 빠르게



물리 서버 시대

~2000년대 초

- 서버 1대 = 애플리케이션 1개
- 자원 활용률 10~20% 수준의 낭비
- 서버 증설에 수 주 소요



가상머신(VM) 시대

2000년대~

- 하이퍼바이저로 하드웨어를 가상화
- 서버 1대에 여러 OS를 격리 실행
- 자원 활용률 대폭 개선, 그러나 무거움



컨테이너 시대

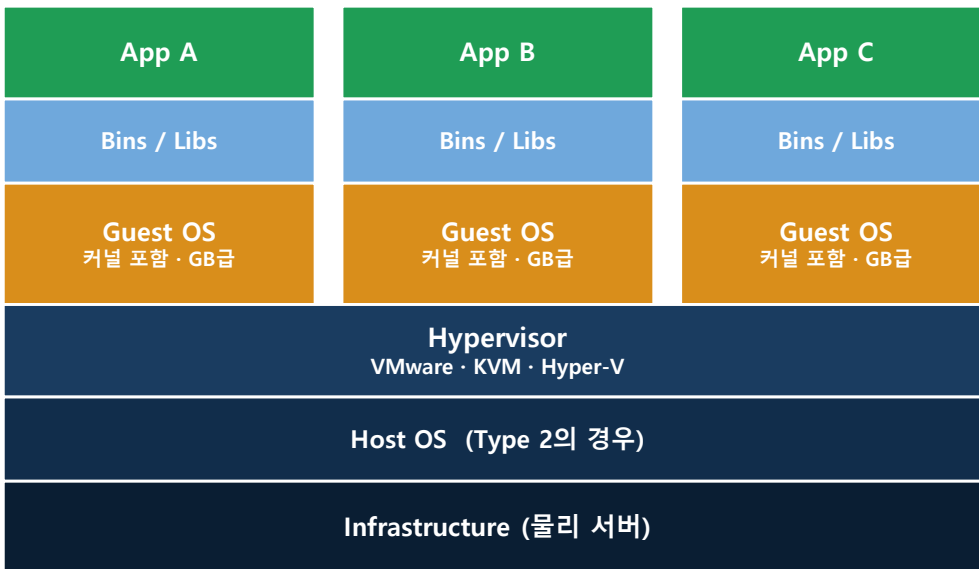
2013년(Docker)~

- OS 커널을 공유하는 프로세스 격리
- 초 단위 시작, MB 단위 크기
- 클라우드·MSA 시대의 표준 배포 단위

핵심 흐름 격리 단위가 '하드웨어 → OS → 프로세스'로 점점 작고 가벼워져 왔다

가상머신(VM)의 구조

하이퍼바이저가 하드웨어를 가상화하여 완전한 컴퓨터를 여러 개 만든다



VM마다 완전한 Guest OS를 별도 탑재

VM의 특징

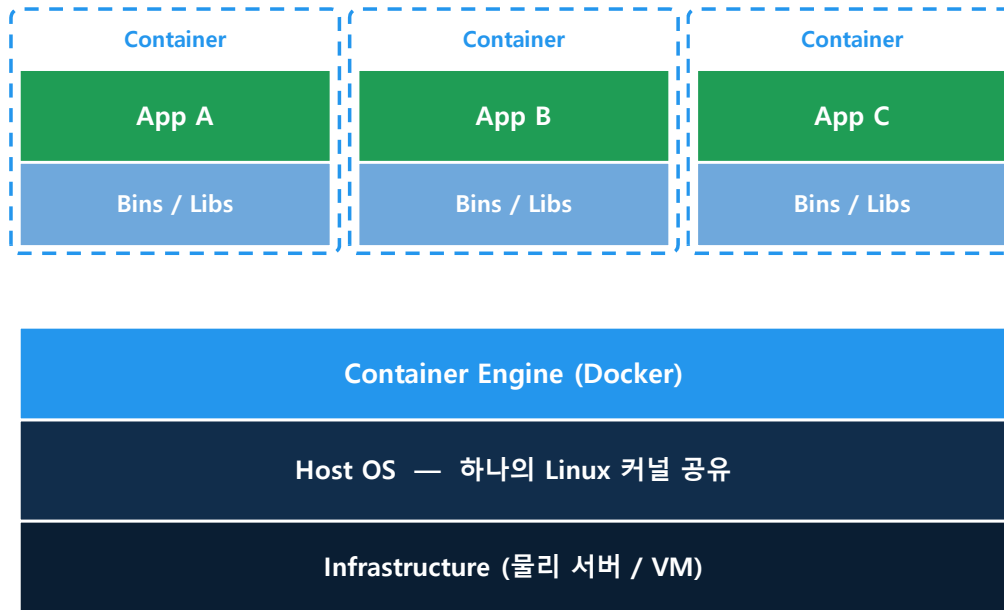
- 하이퍼바이저가 CPU·메모리·디스크를 가상화해 완전한 가상 컴퓨터를 제공
- **VM마다 독립된 Guest OS 커널 → 강력한 격리 수준**
- Guest OS 부팅 필요 → 시작에 수십 초~분 단위
- 이미지 크기 수 GB, VM당 메모리 수백 MB~GB 고정 점유
- Type 1(베어메탈: ESXi, KVM) / Type 2(호스트형: VirtualBox)



장점인 '완전한 격리'가 곧 단점인 '무거움'이 된다 — 앱 하나 띄우려고 OS 하나를 통째로 부팅

컨테이너의 구조

호스트 OS의 커널을 공유하고, 애플리케이션만 격리된 공간에서 실행한다



Guest OS가 없다 — 커널은 호스트 것을 함께 사용

컨테이너의 특징

- **Guest OS 없이 호스트 커널을 공유** → 컨테이너는 사실상 '격리된 프로세스'
- OS 부팅이 없어 초 단위(수십 ms~수 초)로 시작
- 이미지 크기 수 MB~수백 MB — 배포·전송이 가벼움
- 한 서버에 수십~수백 개 실행 가능 (높은 집적도)
- 격리는 커널 기능(namespace, cgroups)으로 구현

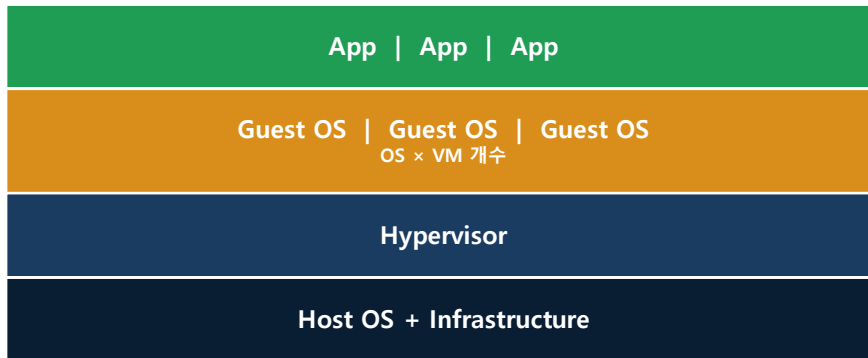


오늘 강의에서 가장 중요한 문장 — "컨테이너는 OS 위의 격리된 프로세스다"

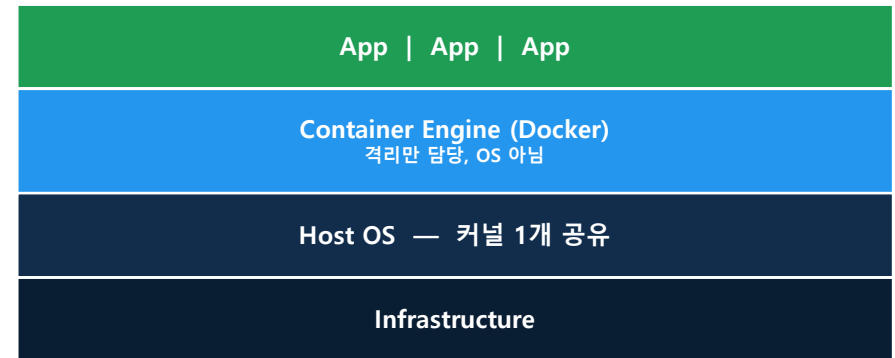
구조 한눈에 비교

결정적 차이는 Guest OS의 유무 — 격리 강도와 무게의 트레이드오프

가상머신



컨테이너



격리의 단위

VM은 하드웨어+OS 전체를, 컨테이너는 프로세스만 격리한다



격리의 강도

VM은 커널까지 분리되어 더 강한 격리, 컨테이너는 커널을 공유한다



무게와 속도

VM은 분 단위 부팅·GB급, 컨테이너는 초 단위 시작·MB급

VM vs 컨테이너 — 항목별 비교

면접에도, 실무 의사결정에도 나오는 핵심 비교표

구분	가상머신 (VM)	컨테이너
가상화 방식	하드웨어 수준 가상화 (하이퍼바이저)	OS 수준 가상화 (커널 기능)
Guest OS	VM마다 필요	없음 — 호스트 커널 공유
시작 시간	수십 초 ~ 분 단위 (OS 부팅)	수십 ms ~ 수 초
이미지 크기	수 GB	수 MB ~ 수백 MB
격리 수준	강함 (커널 분리)	상대적으로 약함 (커널 공유)
집적도(밀도)	서버당 수 개 ~ 수십 개	서버당 수십 ~ 수백 개
OS 다양성	호스트와 다른 OS 가능 (Windows+Linux)	호스트와 같은 커널 계열만
대표 기술	VMware, KVM, Hyper-V, VirtualBox	Docker, containerd, Podman

컨테이너를 만드는 리눅스 커널 기술

Docker가 발명한 것이 아니다 — 리눅스 커널 기능의 영리한 조합



Namespaces

격리 — "보이는 범위"를 분리

- 프로세스마다 독립된 시스템 뷰 제공
- PID·네트워크·마운트·호스트명 등을 각각 분리
- 컨테이너 안에서는 자기 것만 보인다



Cgroups

제한 — "쓸 수 있는 양"을 통제

- Control Groups — 자원 사용량 제한·계량
- CPU, 메모리, 디스크 I/O, 네트워크 대역폭
- 한 컨테이너의 폭주가 전체를 마비시키지 않게



Union FS

효율 — 레이어를 겹쳐 하나로

- 여러 파일시스템 레이어를 겹쳐 하나처럼 마운트
- OverlayFS 등 — 이미지 레이어 구조의 기반
- Part 2의 이미지 레이어에서 다시 만난다

정리 컨테이너 = Namespaces(격리) + Cgroups(제한) + Union FS(레이어)로 포장된 리눅스 프로세스

Namespaces — 무엇을 격리하는가

컨테이너 안의 프로세스는 자신만의 세상을 본다

Namespace	격리 대상	컨테이너에서의 의미
PID	프로세스 ID 공간	컨테이너 안에서는 자기 프로세스가 PID 1부터 시작
NET	네트워크 스택	컨테이너마다 독립된 IP·포트·라우팅 테이블
MNT	마운트 포인트	컨테이너마다 독립된 파일시스템 트리
UTS	호스트명·도메인명	컨테이너가 자기만의 hostname을 가짐
IPC	프로세스 간 통신	공유 메모리·세마포어를 컨테이너 내부로 한정
USER	사용자·그룹 ID	컨테이너의 root ≠ 호스트의 root 매핑 가능



체감 예시

컨테이너 안에서 `ps` 명령을 치면 그 컨테이너의 프로세스만 보이고, 호스트에서 보면 그냥 하나의 일반 프로세스로 보인다



기억할 것

Namespace는 '숨기는' 기술이다. 자원이 없어지는 게 아니라, 보이는 범위를 좁혀서 서로 모르게 만든다

Cgroups — 자원 사용량의 통제

격리(namespace)만으로는 부족하다 — '시끄러운 이웃'을 막아야 한다



CPU

사용 시간·코어 수 제한

`--cpus=1.5`



Memory

최대 메모리 상한 설정

`--memory=512m`



Block I/O

디스크 읽기/쓰기 속도 제한

`--device-read-bps`



Network

대역폭 통제 (tc와 연계)

우선순위 지정

왜 필요한가 — Noisy Neighbor 문제

- 한 서버에 여러 컨테이너가 커널을 공유하며 동거
- 제한이 없으면 한 컨테이너의 메모리 누수·CPU 폭주가 서버 전체를 마비
- **cgroups가 컨테이너별 상한선을 강제해 서로를 보호**

```
# 메모리 512MB, CPU 1개로 제한해 실행
docker run -d --memory=512m --cpus=1 nginx

# 상한 초과 시 → OOM Kill로 컨테이너만 종료
```

VM과 컨테이너 — 대체가 아닌 상호보완

실무에서는 '무엇을 쓸까'가 아니라 '어떻게 조합할까'를 고민한다



VM이 적합한 경우

- 커널 수준의 강한 격리가 필요한 멀티테넌트 환경
- 호스트와 다른 OS 실행 (Linux 위 Windows 등)
- 레거시 시스템, OS 전체 스냅샷·백업이 필요한 워크로드



컨테이너가 적합한 경우

- 마이크로서비스 아키텍처(MSA), 빠른 스케일 아웃
- CI/CD — 빌드·테스트·배포 환경의 일관성
- 높은 집적도가 필요한 클라우드 네이티브 앱

현실의 구성 — 둘을 겹쳐 쓴다

물리 서버 (클라우드 데이터센터)

VM (EC2 인스턴스 등)

컨테이너 (Docker/K8s)

클라우드의 컨테이너는 대부분 VM 위에서 돈다 — 하드웨어 격리는 VM이, 앱 배포 단위는 컨테이너가 담당

PART 2

Docker 아키텍처와 이미지

Docker는 어떻게 구성되고, 이미지와 컨테이너는 무슨 관계인가

- Docker의 등장과 아키텍처
- 이미지 = 읽기 전용 레이어의 묶음
- 이미지와 컨테이너의 관계 (핵심)
- 레지스트리와 이미지 배포

02



 60분

Docker란 무엇인가

컨테이너 기술을 '누구나 쓸 수 있게' 만든 플랫폼

- 2013년 dotCloud(현 Docker Inc.)의 솔로몬 하익스가 공개한 오픈소스 프로젝트
- namespace-cgroups 등 커널 기술은 이전부터 존재 (LXC, chroot 등)
- **Docker의 기여 = 이미지 포맷 + 빌드 도구 + 배포 생태계로 묶어 표준화한 것**
- 이미지 규격은 OCI(Open Container Initiative)로 표준화 — containerd, Podman 등과 호환
- 고래 로고: 컨테이너를 싣고 어디든 운반하는 고래

한눈에 보는 역사

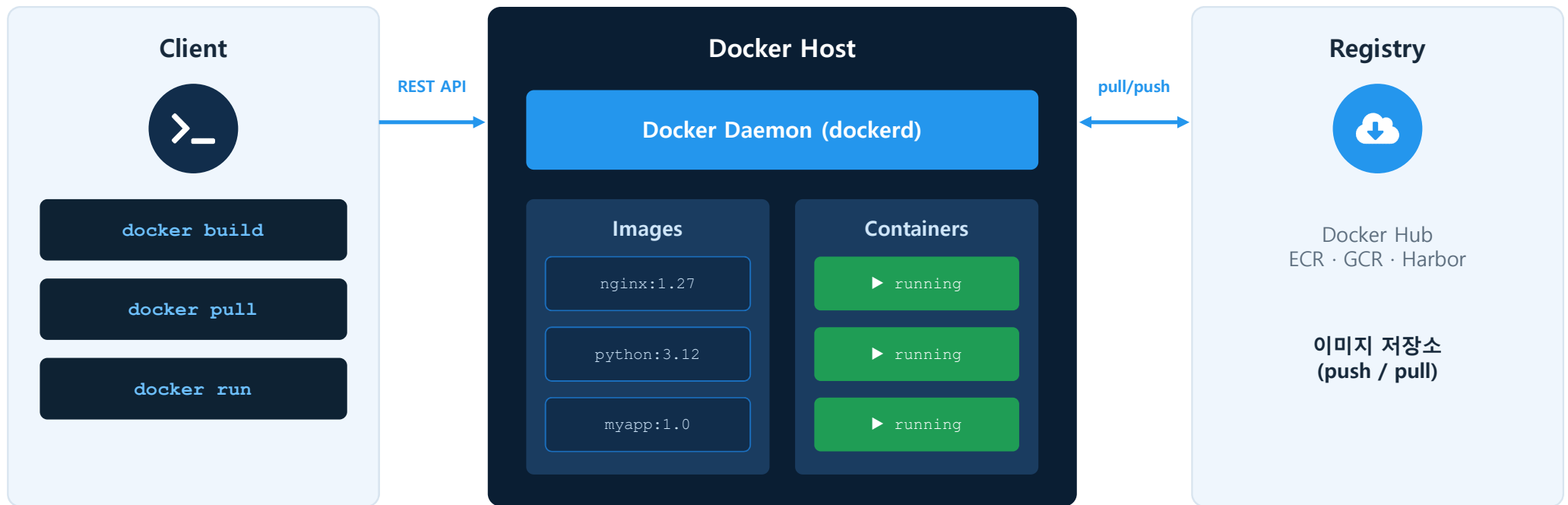
2013	Docker 오픈소스 공개 (PyCon 데모)
2015	OCI 설립 — 이미지·런타임 표준화
2017	containerd를 CNCF에 기부
현재	컨테이너 = 클라우드 배포의 사실상 표준



Docker의 본질 "애플리케이션을 이미지로 만들고(Build), 레지스트리로 나르고(Ship), 어디서든 컨테이너로 실행한다(Run)"

Docker 아키텍처

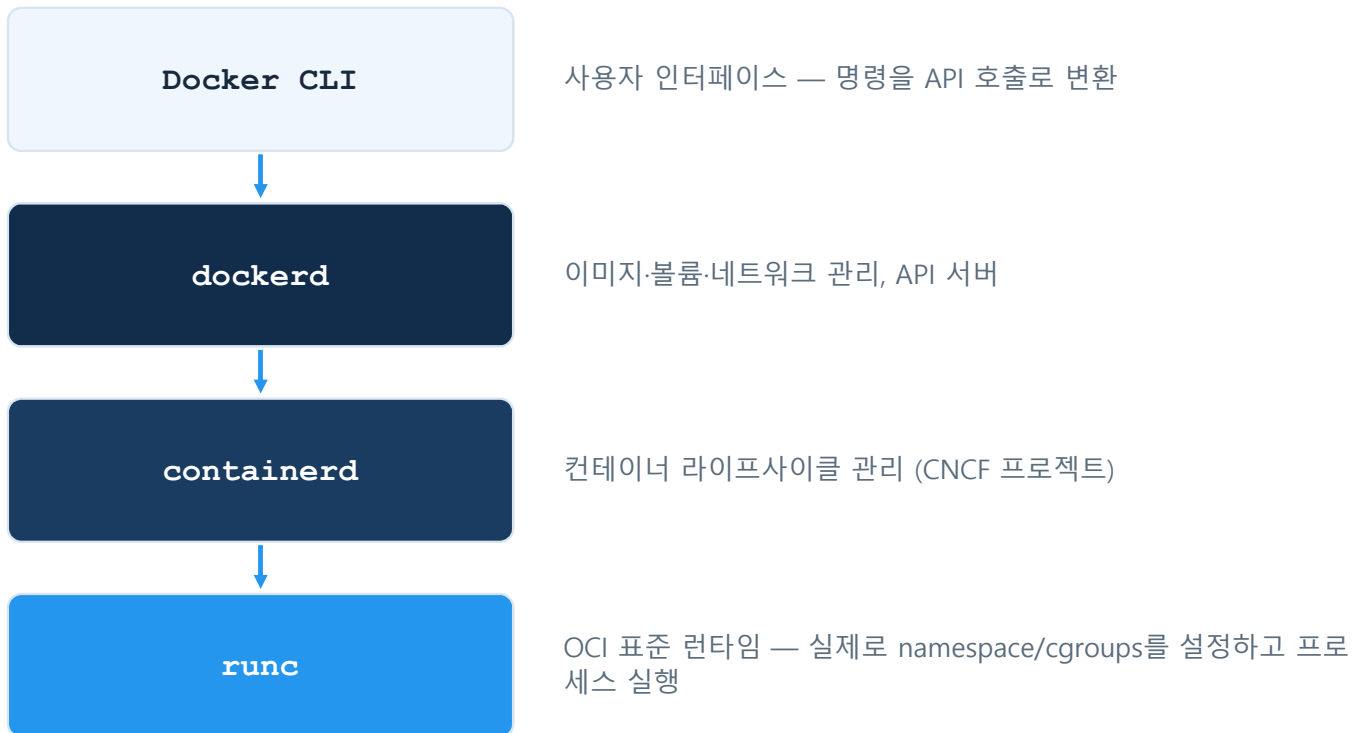
클라이언트-서버 구조: CLI는 명령을 보내고, 데몬이 모든 일을 한다



명령 실행: CLI → (API) → 데몬이 이미지 관리·컨테이너 실행·레지스트리 통신을 모두 담당

Docker 내부 구성요소

dockerd 하나가 아니다 — 표준화된 계층 구조



왜 계층을 나눴나

- 역할 분리 → dockerd 재시작에도 컨테이너는 유지
- OCI 표준 덕분에 런타임 교체 가능



같은 기반의 도구들

- Kubernetes는 containerd를 직접 사용
- Podman·nerdctl 등 대체 CLI도 OCI 호환

Docker 이미지란 무엇인가

컨테이너 실행에 필요한 모든 것을 담은 읽기 전용(Read-Only) 템플릿

이미지에 담기는 것

-  애플리케이션 코드
-  런타임 (Python, Node, JRE ...)
-  시스템 라이브러리·도구
-  환경 변수·설정 파일
-  시작 명령 (무엇을 실행할지)

이미지의 성질

- 불변(Immutable) — 만들어진 뒤에는 내용이 바뀌지 않는다
- 실행되지 않는다 — 실행하는 순간 그것은 '컨테이너'다

비유로 이해하기



밀키트

요리에 필요한 재료·소스·레시피가 포장된 상태. 조리(실행)하면 요리(컨테이너)가 된다



프로그램 설치 파일

실행 파일은 디스크에 있는 정적 존재, 더블클릭해 띄운 프로세스가 컨테이너



클래스와 인스턴스

이미지는 클래스(설계도), 컨테이너는 new로 만든 인스턴스 — 하나의 클래스에서 여러 인스턴스

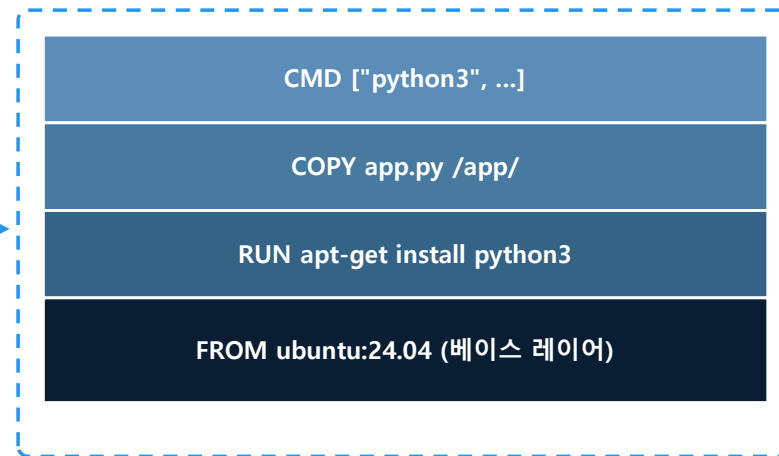
이미지의 레이어 구조

이미지는 한 덩어리가 아니라 — 읽기 전용 레이어를 쌓은 것

Dockerfile (Part 4에서 상세히)

```
FROM ubuntu:24.04  
RUN apt-get install -y python3  
COPY app.py /app/  
CMD ["python3", "/app/app.py"]
```

build



이미지 = 읽기 전용 레이어의 스택

- Dockerfile의 명령 하나가 레이어 하나 — 각 레이어는 '이전 상태와의 차이(diff)'만 저장
- 레이어는 공유된다: ubuntu 베이스를 쓰는 이미지 100개가 있어도 베이스 레이어는 디스크에 1번만 저장
- 변경된 레이어만 다시 빌드·전송 → 빌드 캐시와 빠른 배포의 비결 (Part 4에서 활용)

이미지와 컨테이너의 관계

오늘 강의의 핵심 — 컨테이너는 이미지의 '실행 인스턴스'다

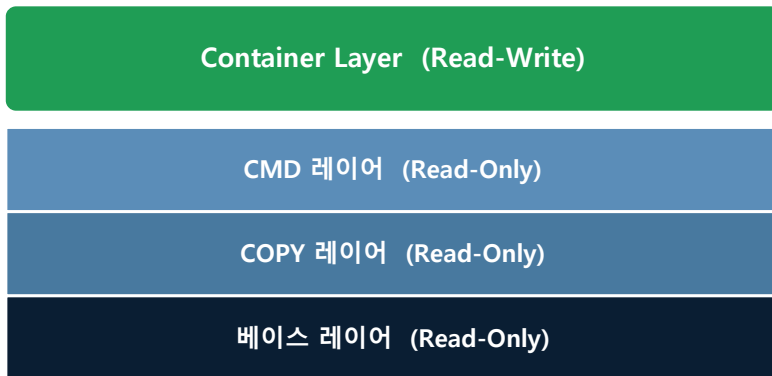


핵심 문장 컨테이너 = 이미지를 실행한 인스턴스. 이미지는 설계도(클래스), 컨테이너는 실체(인스턴스)이며 관계는 1 : N

컨테이너 레이어와 Copy-on-Write

읽기 전용 이미지 위에 '쓰기 가능한 얇은 층' 하나를 얹은 것이 컨테이너

↑ 컨테이너 생성 시 추가되는 유일한 층



이미지 (모든 컨테이너가 공유)

Copy-on-Write(CoW) 동작

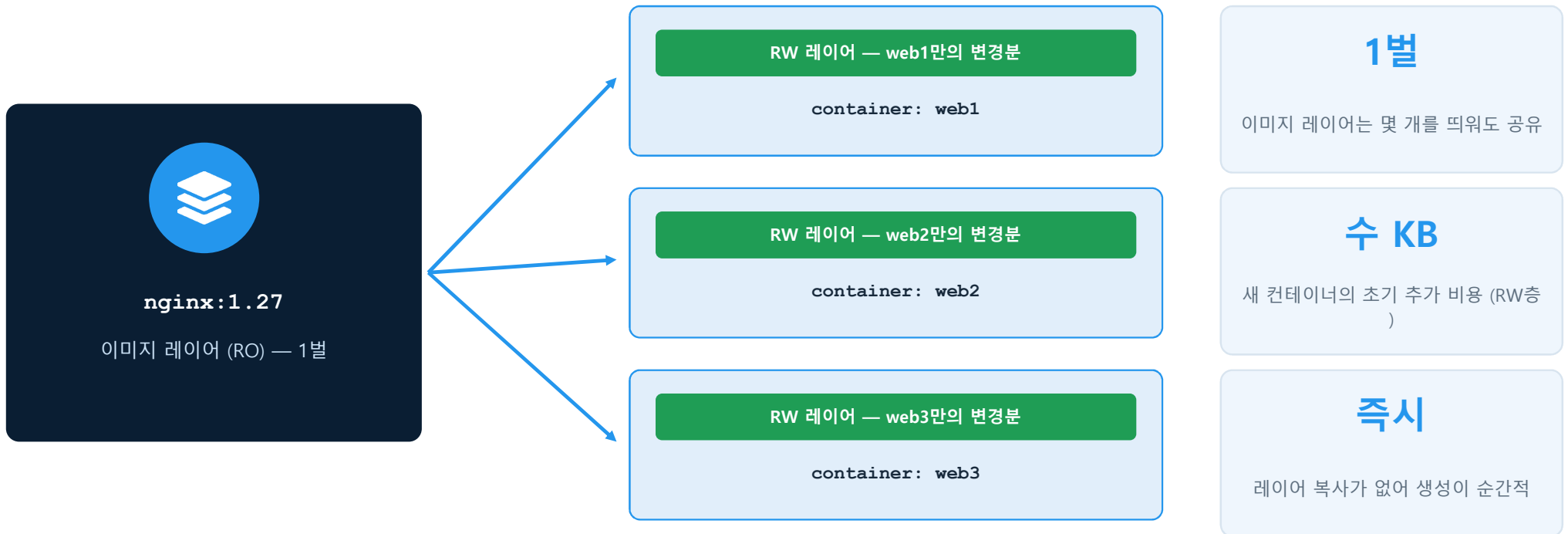
- 읽기 — 아래 이미지 레이어에서 그대로 읽는다 (복사 없음)
- 쓰기·수정 — 해당 파일을 컨테이너 레이어로 복사한 뒤 수정
- 삭제 — 실제로 지우지 않고 컨테이너 레이어에 '가림 표시'만 기록



중요한 결론 컨테이너에서 변경한 모든 데이터는 컨테이너 레이어에만 존재한다.
컨테이너를 삭제하면 그 변경분도 함께 사라진다 → 영속 데이터는 볼륨에 저장해야 (Part 5)

하나의 이미지, 여러 컨테이너

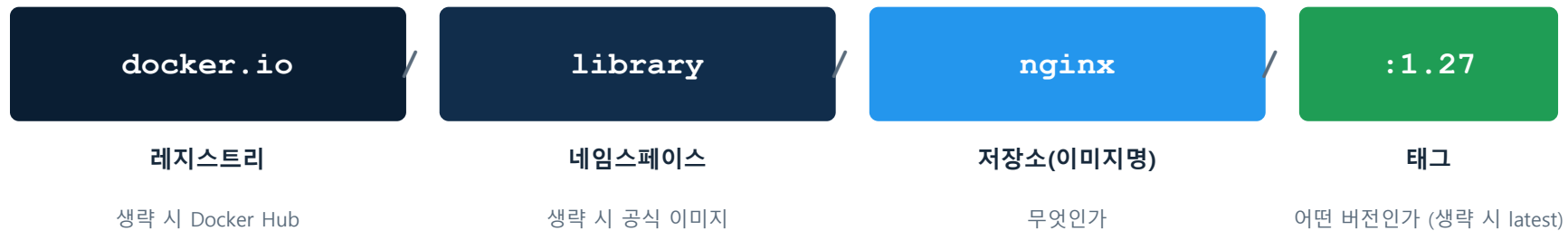
레이어 공유 덕분에 컨테이너 100개를 띄워도 디스크는 거의 늘지 않는다



같은 설계도에서 나온 인스턴스라도 각자의 RW 레이어 덕분에 서로 완전히 독립적으로 동작한다

이미지 이름과 태그

이미지의 주소 체계 — 레지스트리 / 저장소 : 태그



```
# 아래 세 명령은 모두 같은 이미지
docker pull nginx
docker pull nginx:latest
docker pull docker.io/library/nginx:latest

# 사실 레지스트리·개인 저장소는 명시
docker pull myregistry.com:5000/team/app:2.1
```

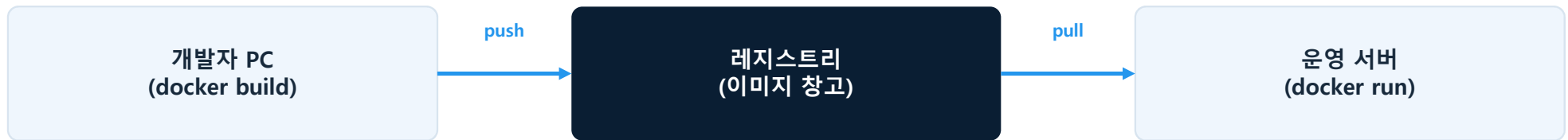


latest의 함정

- latest는 '최신'이 아니라 '태그 생략 시 기본값'일 뿐
- 같은 latest라도 시점에 따라 다른 이미지를 가리킴
- 운영 환경에서는 반드시 버전 태그를 고정할 것

Docker Hub와 레지스트리

이미지를 저장하고 공유하는 창고 — 코드의 GitHub 같은 존재



Docker Hub (hub.docker.com)

- 기본 공개 레지스트리 — pull 명령의 기본 목적지
- **Official Image:** Docker가 검증·관리하는 공식 이미지 (nginx, postgres, python ...)
- Verified Publisher: 벤더가 직접 배포하는 인증 이미지
- 익명 사용자에게는 pull 횟수 제한(rate limit) 존재



프라이빗 레지스트리

- 회사 내부 이미지는 외부 공개 불가 → 사설 창고 사용
- 클라우드 관리형: AWS ECR, Google Artifact Registry, Azure ACR
- 자체 구축: Harbor, GitLab Container Registry
- CI/CD 파이프라인: build → push → 운영 환경에서 pull

이미지 관련 주요 명령어

이미지의 조회 · 다운로드 · 삭제 — 데몬에게 시키는 일들

명령어	동작	자주 쓰는 형태
<code>docker pull</code>	레지스트리에서 이미지 다운로드	<code>docker pull nginx:1.27</code>
<code>docker images</code>	로컬 이미지 목록 (= image ls)	<code>docker images</code>
<code>docker image inspect</code>	이미지 상세 정보(JSON) — 레이어·설정 확인	<code>docker image inspect nginx</code>
<code>docker history</code>	이미지의 레이어 생성 이력 확인	<code>docker history nginx</code>
<code>docker tag</code>	이미지에 새 이름·태그 부여 (별명 붙이기)	<code>docker tag app:1.0 repo/app:1.0</code>
<code>docker rmi</code>	로컬 이미지 삭제 (컨테이너가 쓰는 중이면 실패)	<code>docker rmi nginx:1.27</code>
<code>docker search</code>	Docker Hub 이미지 검색	<code>docker search redis</code>

TIP tag는 복사가 아니라 같은 이미지에 별명을 하나 더 붙이는 것 — 같은 IMAGE ID를 확인해 보라

PART 3

컨테이너 라이프사이클

컨테이너의 탄생부터 소멸까지 — 상태와 명령어

- 라이프사이클 상태 전이도
- docker run의 실체
- 시작·중지·일시정지·삭제
- 실행 중 컨테이너 관리 명령어

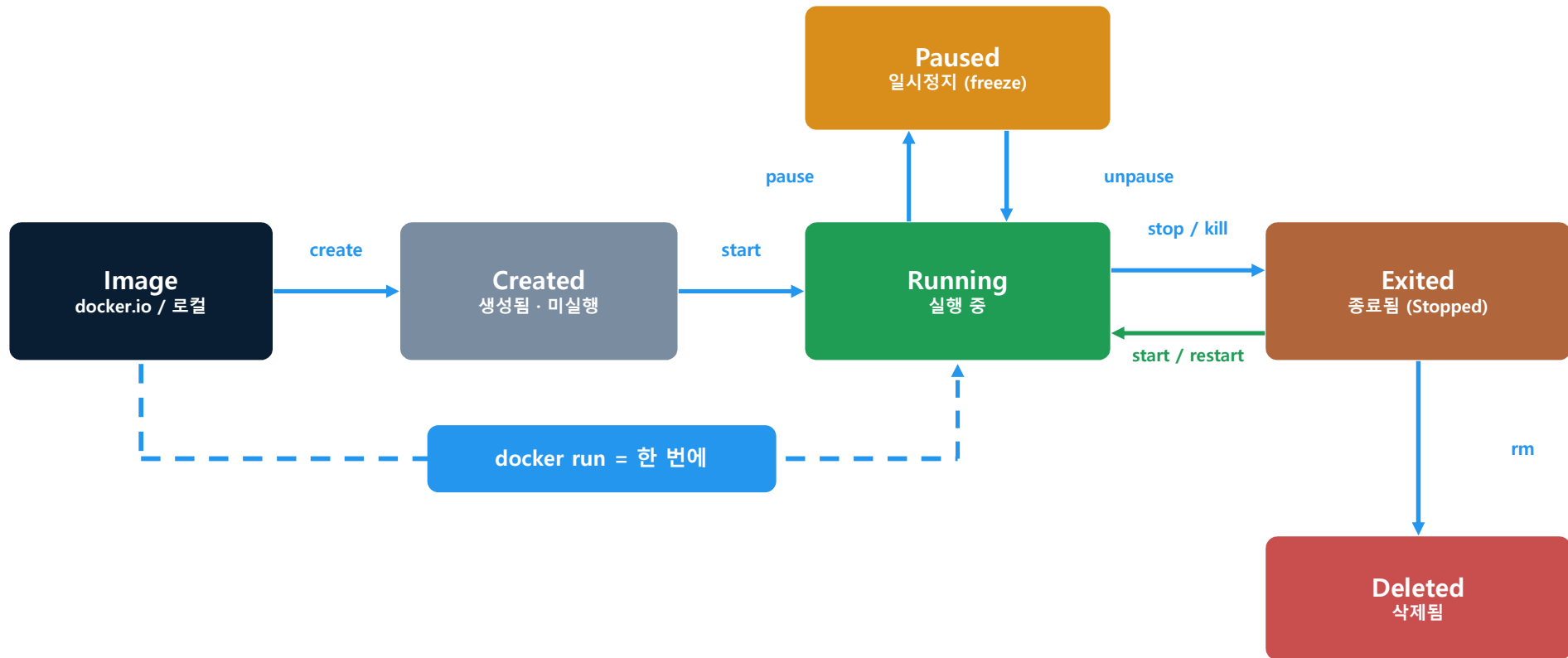
03



 60분

컨테이너 라이프사이클 한눈에 보기

이 그림 하나가 Part 3의 전부 — 상태(박스)와 명령(화살표)



읽는 법 박스 = 컨테이너의 상태, 화살표 = 상태를 바꾸는 docker 명령

docker run의 실체

run은 마법이 아니다 — 여러 명령의 자동 연쇄



```
$ docker run -d --name web -p 8080:80 nginx:1.27
Unable to find image 'nginx:1.27' locally      # ← pull
1.27: Pulling from library/nginx
a2abf6c4d29d: Pull complete                    # ← 레이어별 다운로드
Status: Downloaded newer image for nginx:1.27
3f2c1e8a9b...                                # ← 컨테이너 ID 반환
```



기억할 것

- **run은 항상 '새' 컨테이너를 만든다**
- 멈춘 컨테이너를 다시 켜는 건 start (run 아님)
- 같은 --name으로 run을 두 번 하면 이름 충돌 에러

컨테이너의 상태(Status)

docker ps의 STATUS 컬럼을 읽을 수 있어야 한다

상태	의미	진입 경로
Created	생성됐지만 아직 한 번도 실행되지 않음	docker create
Running	메인 프로세스(PID 1)가 실행 중	run · start · restart
Paused	프로세스가 메모리에 있는 채로 동결(freeze)	docker pause
Exited	메인 프로세스 종료 — 종료 코드 보존	stop · kill · 프로세스 종료
Restarting	재시작 정책에 따라 재기동 중	--restart 정책 동작
Dead	정상 제거 불가능한 비정상 상태 (드뭄)	리소스 오류 등

```
$ docker ps -a
STATUS
Up 2 hours           # Running
Up 5 min (Paused)    # Paused
Exited (0) 1h ago     # 정상 종료
Exited (137) ...      # 강제 종료
```

종료 코드 읽기

- 0 = 정상 종료
- 137 = 128+9, SIGKILL (OOM·kill)
- ps는 Running만, ps -a는 전체 표시

핵심 컨테이너는 메인 프로세스(PID 1)가 살아있는 동안만 Running이다

시작과 중지 — Graceful Shutdown

stop은 '정중한 종료 요청', kill은 '즉시 사살'

docker stop의 동작 순서



명령	동작	언제 쓰나
<code>docker start</code>	Exited 컨테이너를 다시 실행 (데이터·설정 유지)	멈춘 컨테이너 재가동
<code>docker stop</code>	SIGTERM → (10초) → SIGKILL	정상 종료 (기본 선택)
<code>docker restart</code>	stop + start를 한 번에	설정 반영·재기동
<code>docker kill</code>	즉시 SIGKILL (유예 없음)	응답 없는 컨테이너 강제 종료



실무 포인트

- 앱이 SIGTERM을 처리하도록 만들어야 무중단 배포가 가능
- DB 컨테이너에 kill을 쓰면 데이터 손상 위험
- `--restart=always`로 자동 재시작 정책 설정 가능

pause와 stop은 무엇이 다른가

동결(freeze)과 종료(terminate)의 차이



docker pause — 동결

- **cgroup freezer**로 프로세스에 CPU 할당을 중단
- 프로세스는 죽지 않고 메모리에 그대로 유지
- unpause 하면 멈춘 지점부터 즉시 계속 실행
- 메모리는 계속 점유, CPU만 반납
- 용도: 일시적 자원 양보, 순간 상태 보존



docker stop — 종료

- **SIGTERM/SIGKILL**로 프로세스를 종료
- 메모리 해제 — 프로세스 상태는 사라짐
- start 하면 프로세스를 처음부터 다시 시작
- RW 레이어(파일 변경분)는 컨테이너가 남아있는 한 유지
- 용도: 일반적인 중지·재기동



비유 pause는 동영상 일시정지(장면 유지), stop은 플레이어 종료(다시 켜면 처음부터). 단, 파일 변경분은 stop 후에도 남는다는 점이 중요

컨테이너 삭제

rm은 컨테이너의 RW 레이어를 지운다 — 이미지는 그대로

Exited 상태의 컨테이너 삭제

```
docker rm web1
```

실행 중이면 실패 → 강제 삭제 (-f = kill + rm)

```
docker rm -f web1
```

멈춘 컨테이너 일괄 정리

```
docker container prune
```



삭제되면 사라지는 것

- RW 레이어의 모든 파일 변경분
- 컨테이너 설정 (이름, 포트 매핑 등)
- 로그 (docker logs로 보던 내용)
- 이미지는 삭제되지 않는다 (rmi는 별도)



'가축 vs 애완동물' 철학 컨테이너는 아껴 고치는 존재가 아니라, 언제든지 지우고 새로 띄우는 일회용 실행 단위로 다룬다. 그래서 --rm 옵션(종료 시 자동 삭제)이 일회성 작업에 즐겨 쓰인다.

docker run 주요 옵션

run 명령의 옵션이 곧 컨테이너의 '실행 계약서'

옵션	의미	예시
-d	백그라운드(detached) 실행	<code>run -d nginx</code>
-it	대화형 터미널 연결 (셸 사용 시)	<code>run -it ubuntu bash</code>
--name	컨테이너 이름 지정 (미지정 시 랜덤 이름)	<code>--name web</code>
-p	포트 공개: 호스트포트:컨테이너포트	<code>-p 8080:80</code>
-e	환경 변수 주입	<code>-e DB_HOST=db</code>
-v	볼륨/마운트 연결 (Part 5)	<code>-v data:/var/lib/mysql</code>
--rm	종료 시 컨테이너 자동 삭제	<code>run --rm python:3 python -V</code>
--restart	재시작 정책 (no/on-failure/always)	<code>--restart=always</code>

종합 예시 `docker run -d --name web -p 8080:80 -e TZ=Asia/Seoul --restart=always nginx:1.27`

실행 중인 컨테이너 다루기

관찰(logs-stats)과 개입(exec) — 운영의 기본기



docker ps

실행 중 컨테이너 목록 (-a: 전체)



docker logs

표준 출력 로그 확인 (-f: 실시간)



docker exec

실행 중 컨테이너 안에서 새 명령 실행



docker stats

CPU·메모리 사용량 실시간 모니터링



docker inspect

설정·상태 전체를 JSON으로 조회



docker cp

호스트 ↔ 컨테이너 파일 복사

```
# 가장 많이 쓰는 조합 - 컨테이너 쉘 접속과 실시간 로그
docker exec -it web bash          # 컨테이너 안에서 쉘 열기
docker logs -f --tail 100 web     # 최근 100줄부터 실시간 추적
```

exec와 attach의 차이

'새 문을 열고 들어가기' vs 'PID 1에 직접 붙기'



docker exec — 새 프로세스 실행

- 컨테이너 안에 별도의 새 프로세스를 띄운다
- exec로 연 bash를 종료해도 컨테이너는 계속 동작
- 디버깅·점검의 표준 방법
- 예: `docker exec -it web bash`



docker attach — PID 1에 연결

- 메인 프로세스의 입출력에 터미널을 직접 연결
- **Ctrl+C 하면 메인 프로세스가 죽어 컨테이너 종료 위험**
- 분리 단축키: Ctrl+P, Ctrl+Q
- 포그라운드 앱 화면을 봐야 할 때만 제한적으로

실무 결론 컨테이너 내부 작업은 exec를 쓴다. attach는 동작을 정확히 이해하기 전까지는 봉인

라이프사이클 명령어 치트시트

Part 3 요약 — 생성부터 삭제까지 명령 흐름

생성·실행

```
docker run -d --name web -p 8080:80 nginx
docker create / docker start web
```

관찰

```
docker ps -a · docker logs -f web
docker stats · docker inspect web
```

개입

```
docker exec -it web bash
docker cp web:/etc/nginx/nginx.conf .
```

중지·재개

```
docker stop web · docker start web
docker pause web · docker unpause web
```

삭제·정리

```
docker rm web · docker rm -f web
docker container prune · docker rmi nginx
```

PART 4

Dockerfile과 이미지 빌드

남의 이미지를 쓰는 사람에서, 내 이미지를 만드는 사람으로

- Dockerfile — 코드로 정의하는 이미지
- 핵심 명령어와 CMD vs ENTRYPOINT
- 빌드 컨텍스트와 레이어 캐시
- 멀티스테이지 빌드와 경량화

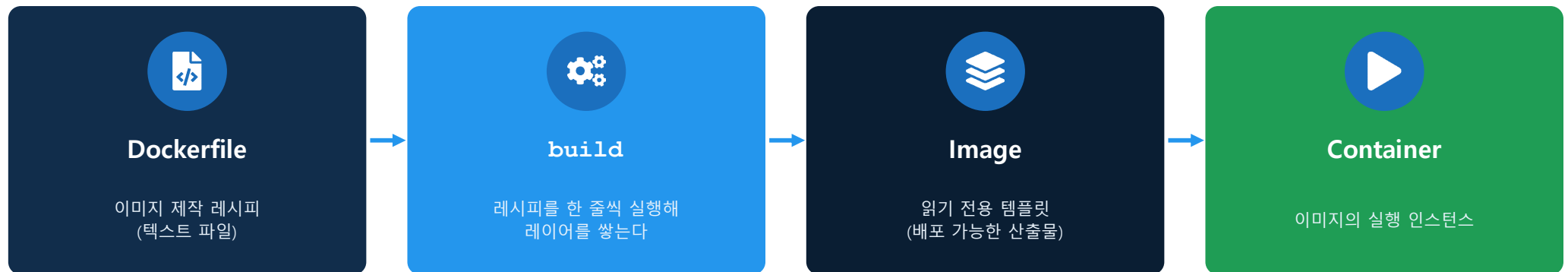
04



 60분

Dockerfile — 코드로 정의하는 이미지

이미지를 만드는 과정 자체를 텍스트 파일로 기록한다



왜 파일로 만드나 — Infrastructure as Code의 이점

- 재현성 — 누가 언제 빌드해도 같은 이미지
- 버전 관리 — Git으로 이미지 제작 과정을 리뷰·추적
- 자동화 — CI가 커밋마다 이미지를 자동 빌드
- 문서화 — 파일 자체가 '이 앱의 실행 환경 명세서'

```
# 최소 Dockerfile
FROM python:3.12-slim
COPY app.py .
CMD ["python", "app.py"]
```

핵심 명령어 ① — 재료 준비

베이스를 고르고, 파일을 넣고, 도구를 설치한다

명령어	역할	메모
FROM	베이스 이미지 지정 — 항상 첫 줄	모든 Dockerfile의 출발점
WORKDIR	이후 명령의 작업 디렉터리 설정	cd 대신 사용 (없으면 생성)
COPY	빌드 컨텍스트의 파일을 이미지로 복사	기본 선택
ADD	COPY + URL 다운로드 + 압축 해제	부작용 많아 특수한 경우만
RUN	빌드 시점에 명령 실행 → 결과가 레이어로	패키지 설치·컴파일 등

```
FROM node:22-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
```



RUN vs CMD 미리보기 RUN은 '빌드할 때' 실행되어 이미지에 구워지고, CMD는 '컨테이너가 시작될 때' 실행된다. 시점이 완전히 다르다 — 다음 슬라이드에서 상세히.

핵심 명령어 ② — 실행 방법 정의

컨테이너가 시작될 때 무엇을 어떻게 실행할지 기록한다

명령어	역할	메모
CMD	컨테이너 시작 시 기본 실행 명령	run 뒤 인자로 통째로 교체 가능
ENTRYPOINT	고정 실행 명령 (교체 불가)	CMD와 조합해 기본 인자 구성
ENV	환경 변수 설정 (빌드·실행 모두 유효)	run -e로 덮어쓰기 가능
ARG	빌드 시점에만 쓰는 변수	--build-arg로 주입
EXPOSE	사용 포트 문서화 (실제 개방은 -p)	메타데이터일 뿐임에 주의
USER	실행 사용자 지정 (root 탈피)	보안 모범 사례

```
ENV APP_ENV=production
ARG VERSION=1.0
EXPOSE 8000
USER appuser
ENTRYPOINT ["python"]
CMD ["app.py"]
```



자주 하는 오해 EXPOSE는 포트를 실제로 열지 않는다. "이 이미지는 8000번을 쓴다"는 문서일 뿐, 실제 개방은 run -p 8080:8000이 한다.

CMD vs ENTRYPOINT

run 뒤에 붙는 인자가 어떻게 해석되는가의 문제



CMD — 통째로 교체되는 기본값

- run 뒤에 인자를 주면 CMD 전체가 교체된다
- "기본 동작 + 자유로운 오버라이드"에 적합



ENTRYPOINT — 고정된 본체

- run 뒤의 인자가 ENTRYPOINT의 '인자'로 붙는다
- "항상 같은 프로그램 + 인자만 바꾸기"에 적합

Dockerfile 설정	docker run myimg	docker run myimg -c 5
CMD ["ping", "localhost"]	ping localhost	-c 5 ← 전체 교체, 실행 실패
ENTRYPOINT ["ping"] CMD ["localhost"]	ping localhost	ping -c 5 ← 인자만 교체

권장 패턴 ENTRYPOINT = 실행 본체(고정), CMD = 기본 인자(교체 가능) 로 조합한다

예제로 보는 Dockerfile 전체 구조

Python 웹 앱의 전형적인 Dockerfile 해부

```
# 1. 베이스 — 검증된 공식 슬림 이미지
FROM python:3.12-slim

# 2. 작업 디렉터리
WORKDIR /app

# 3. 의존성 먼저 (캐시 활용 포인트)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. 소스는 나중에
COPY . .

# 5. 실행 정보
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

1 OS부터 만들지 않는다 — 공식 이미지 위에 얹는다

2 이후 모든 명령의 기준 경로

3 잘 안 바뀌는 의존성을 위쪽에 → 빌드 캐시 재사용

4 자주 바뀌는 소스는 아래쪽에

5 포트 문서화 + 시작 명령

순서에 주목 requirements.txt를 소스보다 먼저 COPY하는 이유는 다음 슬라이드의 '레이어 캐시' 때문

docker build와 빌드 컨텍스트

build 명령의 마지막 인자 '.'의 정체

```
docker build -t myapp:1.0 .  
#           ↳ 태그           ↳ 빌드 컨텍스트 (현재 폴더)  
docker build -f docker/Dockerfile.prod -t myapp:1.0 .
```

- 빌드 컨텍스트 = 데몬에게 통째로 전송되는 폴더
- COPY가 접근할 수 있는 범위가 컨텍스트로 한정됨
- 컨텍스트가 크면 전송부터 느려진다 — node_modules, .git 주의
- .dockerignore로 불필요한 파일을 전송에서 제외

.dockerignore 예시

```
.git  
node_modules/  
*.log  
.env # 비밀번호는 절대 이미지에  
__pycache__/
```



보안 주의

- .env-키 파일이 COPY .. 에 딸려 들어가는 사고가 흔하다
- 이미지에 들어간 비밀번호는 history로 노출될 수 있다

레이어 캐시 — 빌드 최적화의 핵심

바뀐 줄부터 아래만 다시 빌드된다

캐시 동작 규칙

- 각 명령의 결과 레이어는 캐시로 저장된다
- 명령·파일이 그대로면 캐시 재사용 (CACHED 표시)
- 한 줄이 바뀌면 그 줄부터 아래 전부 재실행
- → 자주 바뀌는 명령일수록 아래쪽에 배치

나쁜 순서 — 소스가 바뀔 때마다 pip install 재실행

```
COPY . . # 소스 변경 → 캐시 무효
RUN pip install -r requirements.txt # 매번 재실행!
CMD ["python", "app.py"]
```

체감 효과 (전형적인 Python 앱)

최초 빌드: 수 분

소스만 수정 후 재빌드(캐시 활용): 수 초

명령 순서만 바뀌도 빌드 시간이 수십 배 차이난다

좋은 순서 — 의존성이 안 바뀌면 설치하는 캐시

```
COPY requirements.txt . # 거의 안 바뀜
RUN pip install -r requirements.txt # CACHED ✓
COPY . . # 바뀌는 건 여기부터만
```

멀티스테이지 빌드

빌드 도구는 두고, 실행에 필요한 것만 최종 이미지로

```
# — 1단계: 빌드 전용 스테이지 —  
FROM golang:1.23 AS builder  
WORKDIR /src  
COPY . .  
RUN go build -o /out/server .  
  
# — 2단계: 실행 전용 스테이지 —  
FROM alpine:3.20  
COPY --from=builder /out/server /server  
CMD ["/server"]
```

원리 FROM이 나올 때마다 새 스테이지 시작. COPY --from=이전스테이지 로 산출물만 가져오고, 최종 이미지는 마지막 스테이지만 남는다

이미지 크기 비교 (Go 앱 예시)

단일 스테이지 ≈ 900MB+

멀티스테이지 ≈ 15MB

컴파일러·소스·중간 산출물이 최종 이미지에서 제외



효과

- 작다 — 전송·배포·기동이 빠름
- 안전 — 빌드 도구·소스가 운영 이미지에 없음
- 공격 표면 축소 (보안 스캐너 지적 감소)

이미지 경량화 전략

작은 이미지 = 빠른 배포 + 적은 공격 표면 + 저렴한 저장소



작은 베이스 선택

- python:3.12 (≈1GB) → 3.12-slim (≈150MB)
- alpine: 초경량(5MB)이나 musl 호환성 확인
- distroless: 쉘조차 없는 실행 전용



레이어 다이어트

- RUN 명령은 &&로 묶어 레이어 수 축소
- 설치 후 캐시 삭제를 같은 RUN 안에서
- --no-install-recommends 등 최소 설치



구조적 절감

- 멀티스테이지로 빌드 도구 제거
- .dockerignore로 컨텍스트 정리
- docker history로 큰 레이어 추적

```
# 나쁨: 레이어 3개, 캐시가 이미지에 잔존
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
```

```
# 좋음: 레이어 1개, 캐시까지 정리
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*
```

PART 5

데이터 · 네트워크 · Compose

컨테이너를 '진짜 서비스'로 만드는 마지막 조각들

- 볼륨 — 데이터를 컨테이너 밖에
- 도커 네트워크와 포트 공개
- Docker Compose — 여러 컨테이너를 선언적으로
- 전체 정리와 다음 학습 로드맵

05



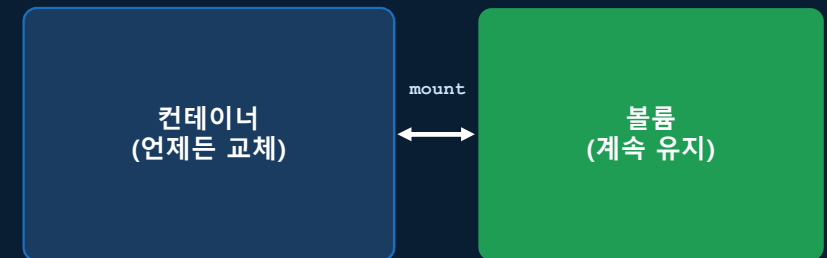
 60분

컨테이너와 데이터의 문제

Part 2의 복선 회수 — RW 레이어는 컨테이너와 운명을 같이한다



해법 — 데이터를 컨테이너 밖으로



- 저장 위치를 컨테이너 수명과 분리
- 컨테이너를 지우고 새로 띄워도 데이터는 그대로
- '가축' 철학을 데이터에도 적용 가능하게 하는 장치

볼륨 · 바인드 마운트 · tmpfs

데이터를 어디에 두느냐 — 세 가지 마운트 방식



Volume

Docker가 관리하는 영역에 저장

- 위치: /var/lib/docker/volumes/...
- Docker가 수명·권한 관리
- DB 등 운영 데이터의 표준

```
-v mydata:/var/lib/mysql
```



Bind Mount

호스트의 특정 경로를 직접 연결

- 위치: 내가 지정한 호스트 폴더
- 호스트 파일을 실시간 반영
- 개발 중 소스 연결에 최적

```
-v $(pwd)/src:/app/src
```



tmpfs

메모리에만 존재 (디스크 X)

- 재시작하면 사라짐
- 임시 파일·민감 정보 캐시
- 리눅스 호스트에서만

```
--tmpfs /app/cache
```

선택 기준 운영 데이터 → Volume · 개발 중 소스 동기화 → Bind Mount · 휘발성 임시 데이터 → tmpfs

볼륨 사용하기

만들고, 연결하고, 확인하고 — 컨테이너를 교체해도 데이터는 산다

```
# 1) 볼륨 생성·조회
docker volume create pgdata
docker volume ls / inspect pgdata

# 2) DB 컨테이너에 연결
docker run -d --name db \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

# 3) 컨테이너 교체 후에도 데이터 유지
docker rm -f db && docker run -d --name db \
  -v pgdata:/var/lib/postgresql/data postgres:17
```



-v 문법 읽는 법

- -v 볼륨이름:컨테이너내부경로
- 볼륨이 없으면 자동 생성
- 경로(/./ 시작)면 바인드 마운트로 해석



관리 명령

- docker volume ls — 목록
- docker volume rm — 삭제
- docker volume prune — 미사용 정리
- rm 컨테이너 ≠ rm 볼륨 (별도 삭제)

Docker 네트워크 기초

컨테이너는 어떻게 서로, 그리고 바깥과 통신하는가



bridge (기본)

- 호스트 안의 가상 스위치(docker0)에 연결
- 컨테이너마다 사설 IP 부여 (172.17.x.x)
- 외부 공개는 -p 포트 매핑으로



host

- 호스트의 네트워크를 그대로 사용
- 격리 없음 — 포트 매핑 불필요/불가
- 고성능·특수 용도 (리눅스 전용)



none

- 네트워크 없음 — 완전 격리
- 루프백(localhost)만 존재
- 배치 작업·보안 격리 용도

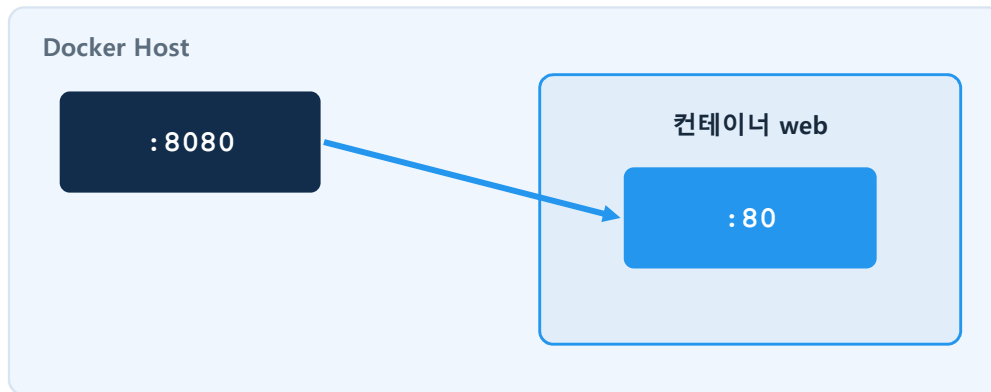


NET namespace의 실체 Part 1에서 본 네트워크 격리가 여기서 구현된다. 각 컨테이너는 자기만의 IP·포트 공간을 갖고, 기본 bridge 네트워크의 가상 스위치를 통해 서로 연결된다.

컨테이너 간 통신과 포트 공개

안에서는 이름으로, 밖에서는 포트 매핑으로

외부 → 컨테이너: 포트 퍼블리싱 (-p)



-p 8080:80 → http://호스트IP:8080 접속 시 컨테이너의 80으로 전달

컨테이너 ↔ 컨테이너: 사용자 정의 네트워크 + DNS

```
docker network create mynet
docker run -d --name db --network mynet postgres:16
docker run -d --name app --network mynet myapp:1.0

# app에서 'db'라는 이름으로 바로 접속
DB_HOST=db (IP를 몰라도 된다)
```



실무 규칙 기본 bridge 대신 '사용자 정의 네트워크'를 만들어 쓰자 — 컨테이너 이름이 곧 DNS 주소가 되는 자동 이름 해석은 사용자 정의 네트워크에서만 제공된다. 이 원리가 다음 주제인 Compose에서 자동화된다.

여러 컨테이너, 어떻게 관리할까

run 명령의 나열에서 — 한 장의 선언적 설계도로

Before — 명령의 나열

```
docker network create mynet
docker volume create pgdata
docker run -d --name db --network mynet \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=... postgres:16
docker run -d --name app --network mynet \
  -p 8000:8000 -e DB_HOST=db myapp:1.0
# 순서 암기, 오타 위험, 팀 공유 어려움
```

After — 파일 하나 + 명령 하나

```
# compose.yaml 에 전체 구성을 선언
services:
  db: { image: postgres:16, ... }
  app: { build: ., ports: [...] }

$ docker compose up -d
# 네트워크 · 볼륨 · 컨테이너 전부 자동 구성
```



선언적

'어떻게'가 아니라 '무엇이 있어야 하는지'를 기술



재현 가능

파일만 공유하면 팀 전체가 같은 환경



한 번에

up 하나로 전체 기동, down 하나로 정리

compose.yaml 해부

지금까지 배운 모든 개념이 이 파일 안에 들어있다

```
services:
  app:
    build: .                # Dockerfile로 빌드
    ports:
      - "8000:8000"        # 포트 퍼블리싱
    environment:
      DB_HOST: db           # 서비스명 = DNS
    depends_on: [db]       # 기동 순서

  db:
    image: postgres:16     # 기존 이미지 사용
    environment:
      POSTGRES_PASSWORD: ${DB_PW} # .env 참조
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:                  # 네임드 볼륨 선언
```

services

컨테이너 1개 = 서비스 1개 (Part 3의 run 옵션들이 항목으로)

build / image

직접 빌드(Part 4) 또는 기존 이미지(Part 2) 선택

ports / environment

run의 -p, -e와 동일 (Part 3)

depends_on

서비스 기동 순서 제어

volumes

데이터 영속성 (Part 5) — 서비스 간 공유 가능

자동 네트워크

같은 Compose의 서비스끼리는 이름으로 통신

Compose 주요 명령어

up과 down — 서비스 전체의 라이프사이클을 한 번에

명령어	동작	메모
<code>docker compose up -d</code>	전체 서비스 빌드·생성·기동	가장 많이 쓰는 명령
<code>docker compose down</code>	컨테이너·네트워크 정리 (볼륨은 유지)	-v 붙이면 볼륨까지 삭제
<code>docker compose ps</code>	서비스 상태 확인	라이프사이클 상태 표시
<code>docker compose logs -f app</code>	서비스별 로그 실시간 확인	서비스명 생략 시 전체
<code>docker compose exec app bash</code>	실행 중 서비스에 명령 실행	docker exec의 Compose판
<code>docker compose build / pull</code>	이미지 갱신	up --build로 한 번에



명칭 참고 과거의 독립 실행형 docker-compose(하이픈, V1)는 지원 종료 — 현재는 Docker CLI에 통합된 docker compose(공백, V2)를 사용한다

WRAP-UP

오늘 배운 것 — 한 장으로 정리

다섯 개의 질문에 답할 수 있다면 오늘 강의는 성공



Part 1

컨테이너 vs VM?

컨테이너는 Guest OS 없이 커널을 공유하는 '격리된 프로세스' — namespace(격리)-cgroups(제한)



Part 2

이미지와 컨테이너의 관계?

이미지는 읽기 전용 레이어의 템플릿, 컨테이너는 그 위에 RW 레이어를 얹은 실행 인스턴스 (1:N)



Part 3

컨테이너의 일생?

run = create+start. Created→Running→Exited→삭제, stop은 SIGTERM→SIGKILL의 정중한 종료



Part 4

이미지는 어떻게 만드나?

Dockerfile로 코드화 — 캐시를 위한 명령 순서, 멀티스테이지로 경량화



Part 5

실제 서비스 구성은?

데이터는 볼륨에, 통신은 사용자 정의 네트워크로, 여러 컨테이너는 Compose로 선언

NEXT STEPS

다음 학습 로드맵

오늘의 기초 위에 쌓아갈 것들



직접 해보기

- 오늘 명령어를 손으로 전부 재현
- Play with Docker (browser 실습장)
- 내 프로젝트를 Dockerfile로 이미지화



CI/CD와 레지스트리

- GitHub Actions로 build & push 자동화
- 프라이빗 레지스트리(ECR-Harbor) 운영
- 이미지 태깅 전략과 버전 관리



보안과 최적화

- 이미지 취약점 스캔 (Trivy 등)
- 비root 실행, 시크릿 관리
- distroless-경량화 심화



오케스트레이션

- 여러 서버로 확장 — Kubernetes
- 서비스 디스커버리·오토스케일링
- Compose 경험이 K8s 이해의 발판

추천 순서 직접 해보기 → 내 프로젝트 컨테이너화 → CI/CD 연결 → Kubernetes 입문



수고하셨습니다!

컨테이너 = 이미지의 실행 인스턴스, 그리고 격리된 프로세스.
이 두 문장을 기억한다면 오늘의 5시간은 성공입니다.

Q & A